

Detection Engineering Program Proposal and Implementation Plan

Operationalizing a Threat-Informed, Governed, and Measurable Detection Engineering Capability

Prepared for	Cybersecurity Leadership Team
Prepared by	Adam Ring
Version	4.0
Date	March 20, 2026

Status Update: Framework established. This document reflects the current-state program design, completed foundation work, and the plan to operationalize and mature the capability.

Internal Use Only

Document Overview

- Executive Summary
- Current State and Progress to Date
- Problem Statement and Need
- Program Goals and Outcomes
- Frameworks, Standards, and Design Principles
- Detection Engineering Lifecycle
- Operating Model and Stakeholder Roles
- Governance and RACI
- Platform Scope and Detection-as-Code Model
- Metrics, KPIs, and Reporting
- Implementation Roadmap
- Year 1 Success Criteria
- Risks and Dependencies
- Executive Decision Request
- Appendices

Executive Summary

Modern adversaries continue to rely on legitimate administrative tools, credential abuse, cloud control-plane activity, living-off-the-land techniques, and stealthy post-compromise behaviors that are not consistently addressed by vendor-default alerting alone.

Detection engineering is the discipline of designing, developing, validating, deploying, and continuously improving detections using structured engineering practices. A mature program improves signal quality, reduces alert fatigue, and provides leadership with measurable visibility into defensive coverage and program maturity.

This document presents an updated proposal and implementation plan for the Detection Engineering Program. The framework has now been established in a centralized repository and is ready to transition from foundational design to sustained operational use.

- A standardized detection lifecycle for intake, research, development, validation, deployment, tuning, and maintenance.
- Threat-informed detection development aligned to MITRE ATT&CK and supporting coverage measurement.
- Governance standards for naming, severity, lifecycle, tagging, and analytic quality.
- Analyst-facing triage guides and reporting structures that connect engineering work to SOC outcomes.
- A Microsoft Sentinel-first implementation model with future expansion potential for Splunk and other platforms.

Current State and Progress to Date

The program is no longer at concept stage. Foundational framework work has been completed and the core operating structure now exists. The current effort should be viewed as a transition from design to operationalization.

The repository now functions as the central source of truth for executive documentation, strategy, process, governance, detection content, triage guides, coverage tracking, and validation support.

Program Deliverables Completed

Area	Current State
Repository foundation	Top-level repository structure established for docs, detections, governance, coverage, content, automation, and tests.
Executive and strategy docs	Program charter, roadmap, maturity model, mission, scope, and supporting executive artifacts created.
Process and governance	Lifecycle, intake, QA, tuning, exception handling, change control, naming, severity, tagging, and analytic quality standards documented.
Detection content	Initial Sentinel content normalized into tactic-aligned folders with increasing standardization and duplicate cleanup underway.
Operational usability	Analyst-facing triage guide structure established and being expanded tactic by tactic.
Reporting and visuals	Metrics catalog, quarterly review artifacts, gap analysis framework, visuals pack, and coverage-related scaffolding created.

Problem Statement and Need

Many organizations still rely heavily on vendor-supplied alerts, ad hoc use-case creation after incidents, and inconsistent analyst feedback loops. The result is uneven coverage, unclear ownership, and a monitoring environment that becomes harder to tune or trust over time.

Without a structured detection engineering capability, common pain points emerge: inconsistent rule quality, unclear promotion criteria, weak validation practices, limited visibility into ATT&CK coverage, and repeated cycles of false positives that burden the SOC.

A formal program addresses these issues by treating detections as governed engineering artifacts rather than one-off searches or isolated alert rules.

- Inconsistent detection quality and metadata make content hard to review and maintain.
- Reactive rule creation leads to uneven coverage and missed opportunities to close known gaps.
- Limited validation evidence increases the risk of noisy or brittle detections entering production.
- SOC feedback often exists informally and is not consistently translated into tuning or lifecycle decisions.

- Leadership lacks a consistent way to understand coverage, maturity, and return on engineering effort.

Program Goals and Outcomes

The Detection Engineering Program should establish a repeatable and scalable approach to creating, validating, tuning, and governing detections. The goal is not simply to increase alert count, but to increase confidence, coverage, and operational value.

- Standardize how detections are requested, developed, reviewed, promoted, and maintained.
- Improve detection fidelity and analyst usability while reducing unnecessary alert noise.
- Provide measurable coverage and maturity reporting for leadership.
- Strengthen collaboration between detection engineering, SOC, incident response, threat intelligence, and validation partners.
- Create a sustainable operating model that can expand across platforms over time.

Frameworks, Standards, and Design Principles

The program should remain threat-informed and standards-aligned. MITRE ATT&CK remains the primary behavior model for mapping adversary techniques and tracking coverage. Other references such as D3FEND, purple-team validation practices, and industry guidance on log management and incident handling provide useful supporting structure.

The operating principle behind the framework is simple: detections should be understandable, measurable, validated, and maintainable. This requires common metadata, clear ownership, lifecycle discipline, and documentation that supports both engineering and analyst use.

- Threat-informed development tied to adversary behaviors and priority risks.
- Detection-as-Code practices with version-controlled content and repeatable review.
- Governance standards that make content easier to scale and report on.
- Validation before promotion whenever practical.
- Continuous improvement through SOC, IR, and exercise-driven feedback.

Detection Engineering Lifecycle

The lifecycle should define how detections move from idea to operational content. Each stage needs a clear purpose and a consistent output so engineering work does not become ad hoc.

- Intake and prioritization – identify new use cases, gaps, requests, or post-incident requirements.
- Research and analysis – understand attacker behavior, telemetry availability, and detection feasibility.
- Development – implement the rule using standardized templates and metadata.
- Validation – confirm the logic against known behavior, simulated activity, or case evidence.
- Deployment – promote content through approved workflow into production monitoring.
- Monitoring and tuning – review fidelity, false positives, and analyst usability.
- Maintenance and retirement – keep content current and deprecate superseded detections cleanly.

Operating Model and Stakeholder Roles

Detection engineering should not operate in isolation. The best results come from a shared operating model where detection engineers own the build-and-maintain workflow, while SOC, threat intelligence, incident response, and adversary simulation functions continuously feed the process with operational insight.

Leadership sponsorship remains critical because the program requires prioritization, decision support, and visible adoption of standards across the broader security organization.

- Detection Engineering owns content development, maintenance, normalization, and quality.
- SOC provides operational feedback, false-positive insight, and usability input.
- Threat Intelligence informs priority adversary behaviors and use-case development.
- Incident Response contributes lessons learned from real intrusions and investigations.
- Red/Purple Team functions provide validation and gap discovery through controlled testing.

Governance and RACI

A lightweight RACI model ensures accountability while preserving the collaboration required for detection development, validation, tuning, and SOC adoption.

Activity	Detection Eng.	SOC	Threat Intel	IR	Red/Purple Team	Leadership
Detection intake and prioritization	R	C	C	C	C	A
Detection development and maintenance	A/R	I	C	C	C	I
Validation and purple-team testing	R	C	C	C	A/R	I
Tuning and analyst usability review	R	A/R	I	C	I	I
Coverage reporting and roadmap review	R	C	C	C	C	A
Change approval for production promotion	R	C	I	C	I	A

R = Responsible | A = Accountable | C = Consulted | I = Informed

Platform Scope and Detection-as-Code Model

The current implementation is Microsoft Sentinel-first. This is the platform around which the initial content model, triage guides, reporting structure, and governance have been organized. That focus is appropriate and should remain explicit in the document.

At the same time, the framework has been designed so that standards, lifecycle, reporting, and governance can expand into future platforms such as Splunk without rebuilding the program from scratch.

- Initial operational platform: Microsoft Sentinel.

- Repository-managed detection content and supporting documentation.
- Templates and metadata designed for future cross-platform reuse.
- Future-state expansion path for Splunk and other security analytics platforms.

Metrics, KPIs, and Reporting

The program should report a concise set of metrics that leadership can understand and engineering can act upon. The intent is to measure coverage, quality, operational value, and program maturity—not just raw alert volume.

Category	Metric	Why It Matters
Coverage	% of priority ATT&CK techniques with mapped detections	Shows whether engineering effort is addressing the adversary behaviors that matter most.
Quality	False positive rate and analyst feedback trend	Measures signal quality and operational trust in detections.
Operations	Average time from intake to validated detection	Shows whether the program is becoming repeatable and efficient.
Validation	% of production detections with test evidence	Confirms detections are not promoted without meaningful validation.
Maturity	# of detections under lifecycle governance	Demonstrates adoption of engineering discipline and sustainable maintenance.
Leadership	Quarterly gap-closure progress	Connects engineering work to roadmap execution and risk reduction.

Implementation Roadmap

The roadmap should now reflect that foundational framework work is largely complete. The next phases focus on operationalization, validation, and sustainable maturity.

Phase	Status	Focus	Key Outcomes
Phase 1	Completed	Foundation and framework	Repository structure, governance standards, templates, lifecycle, reporting scaffolding, visuals, and initial content established.
Phase 2	Current	Operationalization	Normalize priority Sentinel detections, align triage guides, formalize intake and promotion workflow, and embed SOC feedback.
Phase 3	Next	Validation and adversary simulation	Expand test evidence, validate high-priority detections, and use purple-team exercises to close detection gaps.

Phase	Status	Focus	Key Outcomes
Phase 4	Planned	Maturity and multi-platform expansion	Strengthen automation, reporting cadence, and prepare standards for future Splunk or cross-platform adoption.

Year 1 Success Criteria

The first year of operationalization should be measured by a manageable set of concrete outcomes rather than an unrealistic volume target. The objective is to prove repeatability and value, not simply to produce more rules.

- Establish the repository as the authoritative source of truth for program governance and active detection content.
- Normalize priority Sentinel detections into the standardized schema and cleanly deprecate superseded content.
- Publish triage guides for the highest-value detections and align them with SOC workflows.
- Generate quarterly reporting on coverage, quality, and roadmap progress.
- Validate a meaningful subset of high-priority detections with retained evidence.
- Demonstrate measurable reduction in detection sprawl and improvement in content governance.

Risks and Dependencies

The program should acknowledge the dependencies that affect its success. Doing so improves realism and helps leadership understand where support is required.

- Telemetry quality and onboarding gaps can limit what is feasible to detect.
- SOC analyst bandwidth affects tuning feedback and validation support.
- Purple-team or simulation capacity influences how quickly validation can mature.
- Engineering time is needed for documentation, normalization, and maintenance—not only new rule creation.
- Cross-team adoption of standards is required for the program to scale consistently.

Executive Decision Request

Leadership approval is requested to formally recognize the Detection Engineering Program framework and support its transition into routine operational use. The framework is now established; the next step is sustained execution and maturity growth.

- Approve the governance model and lifecycle as the program baseline.
- Support Sentinel-first operationalization and future multi-platform expansion planning.
- Recognize the repository as the official program source of truth for documentation and detection content.
- Support quarterly reporting and roadmap review as part of program oversight.
- Prioritize the cross-functional participation required for tuning, validation, and coverage improvement.

Appendices

Appendix A – Detection Intake Requirements

Each request should capture a concise problem statement, suspected behavior or technique, data-source expectations, business priority, owner, and operational justification.

Requests should be triaged into a consistent intake queue so the team can prioritize based on risk, coverage gap, incident lessons learned, or threat intelligence relevance.

Appendix B – Detection Rule Standard

Production content should follow a consistent Detection-as-Code structure with metadata such as title, identifier, description, platform, query, ATT&CK mapping, severity, false positives, validation notes, lifecycle state, and owner.

Standardization reduces review friction, improves maintainability, and supports reporting, coverage tracking, and future CI/CD validation.

Appendix C – Validation Expectations

High-value detections should have evidence of validation, whether through lab simulation, Atomic Red Team, purple-team activity, replayed telemetry, or case-driven confirmation.

Validation artifacts should be retained in a repeatable way so future tuning or regression testing can reference known-good behavior and expected results.

Appendix D – Change Control and Promotion

Rules should move through lifecycle stages such as experimental, testing, production, and deprecated. Promotion decisions should reflect logic quality, validation evidence, and analyst usability.

Change records should note what changed, why it changed, the expected operational impact, and any rollback considerations.

Appendix E – Repository Alignment

The current framework is implemented in a centralized repository that includes executive documents, strategy, process, governance, detections, triage guides, reporting, coverage, automation, and tests.

This repository structure is not just documentation; it is the operating backbone for a governed detection engineering program.